



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Experiment: 7 Name: Dhanish Ladwani PRN: 24070521020

1. Experiment Title:

Perform DOM traversal and updates; develop a to-do list app with add, edit, and delete features.

2. Software/Tools Required:

Google Chrome (Web Browser), Visual Studio Code (Code Editor), Browser Developer Tools / Console, HTML, CSS Javascript

3. Experiment Program Code:

Experiment 7.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>To Do List App</title>
  <link rel="stylesheet" href="Experiment 7.css">
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&display=swap"
rel="stylesheet">
</head>
<body>
  <div class="container">
    <h2>To Do List</h2>

    <div class="input-group">
      <input type="text" id="taskInput" placeholder="Enter a task...">
      <button id="addBtn">Add</button>
    </div>

    <ul id="taskList"></ul>

    <footer style="text-align: center; margin-top: 30px; font-size: 14px; color: gray; border-top: 1px
solid lightgray; padding-top: 15px;">
      Made by Dhanish Ladwani PRN - 24070521020
    </footer>
  </div>

  <script src="Experiment 7.js"></script>
</body>
</html>
```

Experiment 7.css

```
* {
  box-sizing: border-box;
}

body {
  background-color: whitesmoke;
  color: #333;
  font-family: 'Inter', sans-serif;
```

```
margin: 0;
padding: 40px 20px;
display: flex;
justify-content: center;
}

.container {
width: 100%;
max-width: 500px;
background-color: white;
padding: 30px;
border: 1px solid lightgray;
border-radius: 12px;
box-shadow: 0 4px 15px lightgray;
}

h2 {
color: darkblue;
text-align: center;
margin-top: 0;
margin-bottom: 24px;
font-size: 24px;
}

.input-group {
display: flex;
gap: 10px;
margin-bottom: 20px;
}

input {
flex: 1;
padding: 10px 14px;
border: 1px solid lightgray;
border-radius: 6px;
font-size: 14px;
outline: none;
transition: border-color 0.2s;
}

input:focus {
border-color: royalblue;
}

#addBtn {
padding: 10px 18px;
background-color: royalblue;
color: white;
border: none;
border-radius: 6px;
font-size: 14px;
font-weight: 600;
cursor: pointer;
transition: background-color 0.2s;
}

#addBtn:hover {
background-color: blue;
}

ul {
list-style-type: none;
padding: 0;
margin: 0;
}

li {
display: flex;
justify-content: space-between;
```

```

    align-items: center;
    background-color: whitesmoke;
    border: 1px solid lightgray;
    margin-top: 10px;
    padding: 12px 14px;
    border-radius: 6px;
    word-break: break-word;
}

.task-text {
    font-size: 14px;
    color: #333;
    flex: 1;
    margin-right: 10px;
}

.buttons {
    display: flex;
    gap: 6px;
}

.buttons button {
    padding: 6px 12px;
    font-size: 13px;
    font-weight: 500;
    border-radius: 4px;
    cursor: pointer;
    border: none;
    transition: opacity 0.2s;
}

.buttons button:hover {
    opacity: 0.85;
}

.edit {
    background-color: darkorange;
    color: white;
}

.delete {
    background-color: #d9534f;
    color: white;
}

```

Experiment 7.js

```

const taskInput = document.getElementById('taskInput');
const addBtn = document.getElementById('addBtn');
const taskList = document.getElementById('taskList');

function addTask() {
    let task = taskInput.value.trim();
    if (task === "") {
        alert("Please enter a task!");
        return;
    }

    const li = document.createElement('li');

    const span = document.createElement('span');
    span.className = 'task-text';
    span.textContent = task;

    const buttons = document.createElement('div');
    buttons.className = 'buttons';

    const editBtn = document.createElement('button');

```

```

editBtn.textContent = 'Edit';
editBtn.className = 'edit';
editBtn.addEventListener('click', function() {
    let newTask = prompt("Edit your task:", span.textContent);
    if (newTask !== null && newTask.trim() !== "") {
        span.textContent = newTask.trim();
    }
});

const deleteBtn = document.createElement('button');
deleteBtn.textContent = 'Delete';
deleteBtn.className = 'delete';
deleteBtn.addEventListener('click', function() {
    taskList.removeChild(li);
});

buttons.appendChild(editBtn);
buttons.appendChild(deleteBtn);

li.appendChild(span);
li.appendChild(buttons);
taskList.appendChild(li);

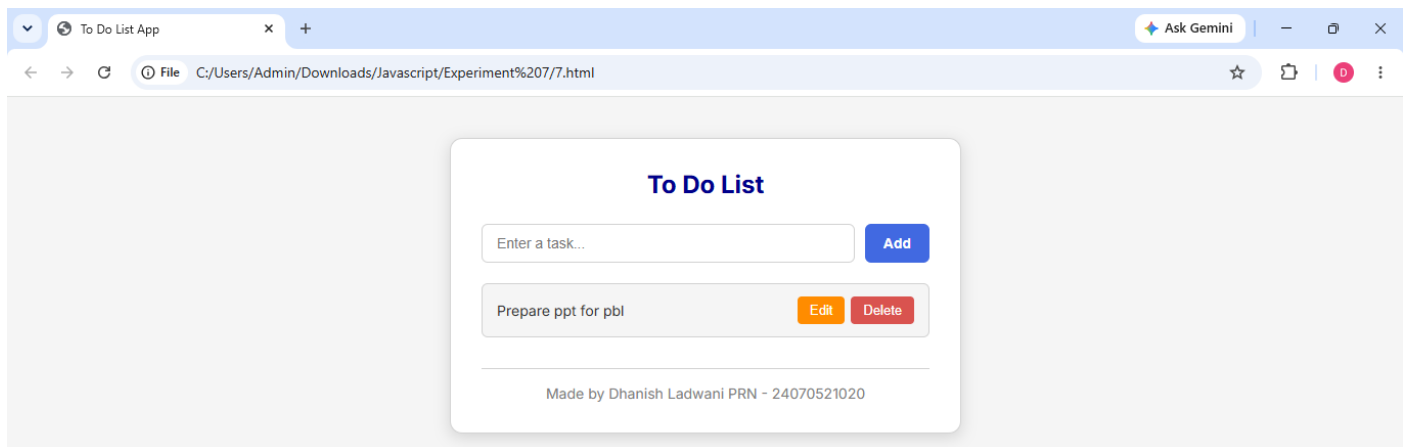
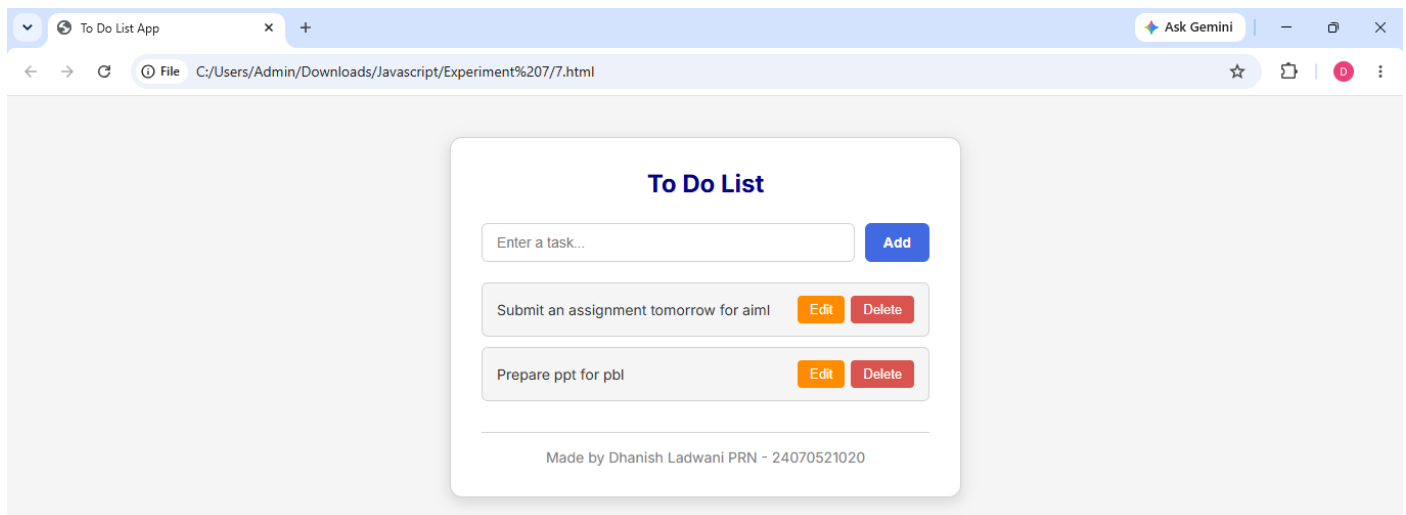
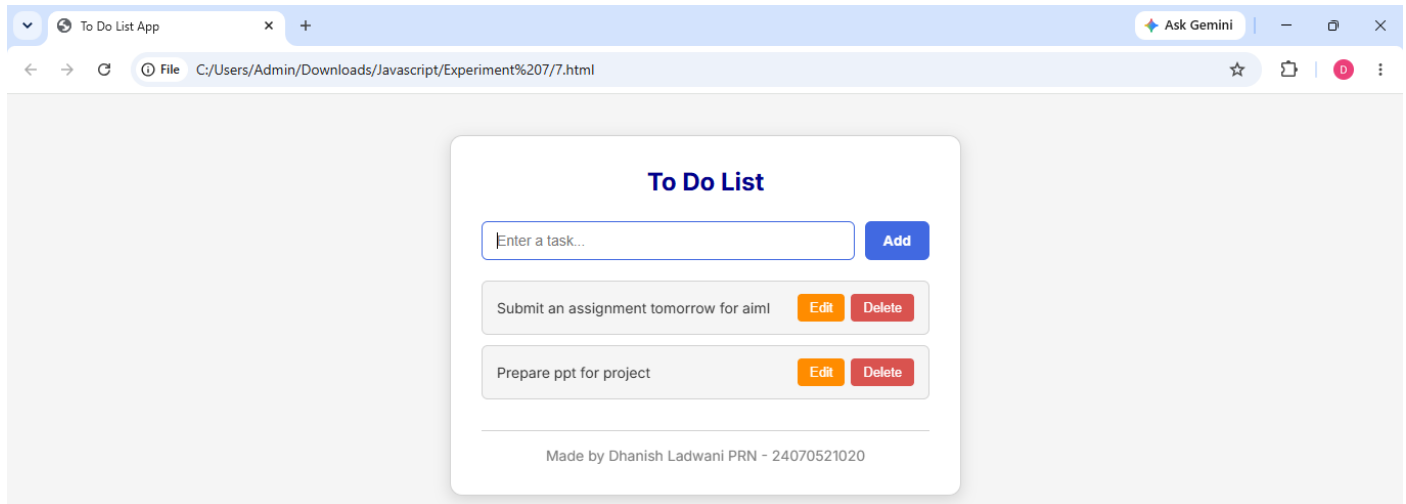
taskInput.value = "";
taskInput.focus();
}

addBtn.addEventListener('click', addTask);

taskInput.addEventListener('keypress', function(event) {
    if (event.key === 'Enter') {
        addTask();
    }
});

```

4. Output:



5. Result/Conclusion:

DOM traversal and dynamic element creation were demonstrated by building a to-do list application. New tasks are added by creating list elements with `createElement()` and appending them with `appendChild()`, and each task can be removed using `removeChild()`. This reinforced dynamic DOM manipulation and event handling in JavaScript.